Informe Técnico de Laboratorio: ARP Spoofing y Session Hijacking (Man-in-the-Middle Combinado).

Este informe documenta los resultados técnicos, la metodología de ejecución, los indicadores de compromiso (IoCs) y las contramedidas defensivas de la práctica de **ARP Spoofing y Session Hijacking** . El ejercicio se desarrolló en un entorno virtualizado controlado compuesto por Kali Linux como estación de auditoría, Metasploitable 2 como servidor objetivo (alojando la aplicación web DVWA) y un entorno de red local aislado (LAN).

1. Contexto Técnico y Arquitectura del Laboratorio.

El análisis de tráfico *Adversary-in-the-Middle* (AiTM) o *Man-in-the-Middle* (MitM) mediante la manipulación de la tabla de resolución de direcciones (ARP) constituye una de las amenazas más relevantes en redes de área local. Al interceptar el flujo de comunicaciones no cifradas en la Capa 2 (Enlace de Datos) y Capa 7 (Aplicación), se pueden observar tokens de autenticación activos y analizar cómo se reutilizan para acceder a sesiones sin requerir credenciales de acceso.
Los objetivos principales de este informe se orientan a la formación del analista defensivo en el Centro de Operaciones de Seguridad (SOC), cubriendo desde la redirección de la capa de enlace hasta la validación de controles de desarrollo web seguro.

2. Fase 1: Habilitación de Reenvío IP y Envenenamiento ARP (arpspoof).

La fase inicial del ejercicio requiere posicionar la interfaz de red de Kali Linux entre la máquina auditada y la puerta de enlace. Para garantizar que el tráfico continúe fluyendo sin generar interrupciones en el servicio, el kernel del sistema operativo debe actuar como enrutador transparente.

2.1. Habilitación del reenvío de paquetes en el kernel

Por defecto, los sistemas operativos Linux descartan los paquetes IP recibidos cuya dirección de destino no coincida con sus propias interfaces. Para activar la redirección de paquetes a nivel de kernel, se ejecuta la instrucción:

```
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
```

El valor 1 confirma la activación del reenvío en la pila de red IPv4.

2.2. Ejecución de respuestas ARP modificadas bidireccionales

El protocolo ARP (Address Resolution Protocol) carece de mecanismos de autenticación intrínsecos y acepta respuestas ARP Reply no solicitadas (*Unsolicited ARP Replies*). La herramienta arpspoof emite ráfagas de paquetes para actualizar las tablas de caché de ambos extremos.
Se ejecutan dos procesos simultáneos en terminales independientes:

```
# Terminal 1: Actualización de la tabla ARP de la víctima (asocia la IP del Gateway a la MAC de Kali)
sudo arpspoof -i <eth0-1> -t <IP_VICTIMA> <IP_GATEWAY>

# Terminal 2: Actualización de la tabla ARP del Gateway (asocia la IP de la víctima a la MAC de Kali)
```

```
sudo arpspoof -i <eth0-1> -t <IP_GATEWAY> <IP_VICTIMA>
```

La consola mostrará un flujo continuo de mensajes confirmando el reenvío de identidades físicas en la Capa 2.

🧠 Sin rodeos:

¿Qué se hizo exactamente en la Fase 1?

Imagínate que la máquina víctima y el router son dos personas en la misma oficina que se envían cartas constantemente. Para saber a quién entregarle cada paquete, ambas revisan su libreta de contactos (la **tabla ARP**), donde anotan el nombre/IP y la dirección física/MAC de cada compañero.
Lo que se logró al ejecutar los comandos de arpspoof fue redirigir la correspondencia:

1. Le dijimos a la víctima: *"Hola, yo soy el router, así que entrégame todas tus cartas a mí"* .
2. Le dijimos al router: *"Hola, yo soy la víctima, así que envíame todas sus respuestas a mí"* .

Con el comando ip_forward = 1, convertimos a Kali Linux en un "cartero invisible". En lugar de detener la carta, Kali la recibe, toma nota en silencio y la entrega de inmediato a su destino real. Así, la máquina auditada sigue navegando sin notar interrupciones.

¿Por qué importa para el equipo de seguridad (Blue Team)?

Importa porque demuestra que el protocolo ARP es confiado por diseño: **acepta respuestas sin pedir autenticación** . Esta posición intermedia es el requisito previo para poder analizar el tráfico no cifrado que transita por la red.

3. Fase 2: Captura e Inspección de Tráfico Web (http.cookie) en Wireshark.

Con la posición de intermediario consolidada, todo el tráfico generado entre la máquina auditada y la aplicación web transita a través de la interfaz de red de Kali Linux.

3.1. Inspección de paquetes y filtrado de telemetría HTTP

Al acceder a una aplicación web que opera sobre el protocolo HTTP no cifrado (puerto 80 TCP) e iniciar sesión, (EJ: DVWA), el servidor asigna una cookie de sesión para mantener el estado de autenticación.
Para aislar esta información en Wireshark:

1. Se inicia la captura en la interfaz eth: sudo wireshark -i <eth0-1> &.
2. Se aplica el filtro de visualización específico en la barra superior:

```
http.cookie
```

3.2. Extracción del token de sesión

Al desplegar el paquete capturado en el panel de detalles (Hypertext Transfer Protocol -> Cookie), se observa la cabecera transmitida en texto claro:

```
Cookie: PHPSESSID=[CADENA_TOKEN_SESION]
```

Utilizando la opción **Copy -> Value** del menú contextual sobre la línea de la cookie, se extrae la cadena alfanumérica limpia que representa la identidad temporal de la sesión activa.

🧠 Sin rodeos:

¿Qué se hizo exactamente en la Fase 2?

Imagina que cuando un usuario entra a una zona de acceso controlado (la aplicación web), la recepción le pone una **pulsera con un código único** (la cookie de sesión) para que no tenga que mostrar sus credenciales cada vez que cambie de sección.
Como en la Fase 1 nos pusimos en la ruta del tráfico (MitM), cada vez que el usuario navega por la página, el paquete pasa por nuestras manos. Como el tráfico viaja en un formato no cifrado (HTTP), con la lupa de Wireshark (http.cookie) solo tuvimos que **leer el código escrito en su pulsera e identificarlo** .

¿Por qué importa para el equipo de seguridad (Blue Team)?

Importa porque demuestra que si una aplicación web no cifra su comunicación mediante HTTPS, **el control de acceso no sirve de nada una vez que el usuario inicia sesión** . El token que valida la identidad viaja expuesto en la red a la vista de cualquier analizador de tráfico.

4. Fase 3: Inyección de Token y Secuestro de Sesión.

El secuestro de sesión se concreta al reutilizar el token interceptado en un navegador independiente, comprobando cómo se puede acceder al sistema eludiendo el formulario de autenticación.

4.1. Procedimiento en las herramientas de desarrollo (DevTools)

Para validar la vulnerabilidad desde la perspectiva defensiva:

1. Se abre una ventana privada en el navegador Firefox de Kali Linux.
2. Se navega a la URL de inicio de sesión objetivo de la aplicación web auditada.
3. Se abren las herramientas de inspección (**F12**) y se accede a la pestaña **Almacenamiento (Storage)** -> **Galletas (Cookies)** .
4. Se sustituye el campo **Valor (Value)** de la variable PHPSESSID pegando la cadena alfanumérica extraída en la Fase 2.
5. Se modifica la barra de direcciones apuntando a la ruta principal de la aplicación (index.php) y se presiona **Enter** .

4.2. Resultado de la prueba

El servidor web procesa la solicitud entrante, valida la cookie de sesión en su base de datos activa y devuelve la interfaz con el mensaje de bienvenida del panel administrativo, otorgando acceso a la cuenta autenticada.

🧠 Sin rodeos:

¿Qué se hizo exactamente en la Fase 3?

Con la clave de la pulsera que anotamos en la Fase 2, fuimos a las herramientas del navegador, colocamos **un código idéntico** (inyectamos la cookie con F12) y caminamos directamente hacia la puerta de entrada de la aplicación.
El servidor web no nos pidió usuario ni contraseña; solo leyó el código de la pulsera, reconoció la sesión activa y nos permitió acceder con todos los privilegios del usuario original.

¿Por qué importa para el equipo de seguridad (Blue Team)?

Importa porque evidencia que las contraseñas no son el único factor de seguridad. Si un tercero obtiene el token de sesión activa, la autenticación inicial queda anulada. Esto exige que las aplicaciones web controlen la caducidad de las cookies y las aseguren con atributos defensivos.

5. Fase 4: Análisis SOC, Detección e Indicadores de Compromiso (IoCs).

El analista defensivo debe estar capacitado para detectar las firmas dejadas por esta actividad tanto en los sistemas locales como en los registros del servidor.

5.1. Identificación de la anomalía en la tabla ARP (arp -a)

Al ejecutar el comando de auditoría de tablas de resolución de direcciones en la máquina auditada o en hosts monitoreados:

```
arp -a
```

La consola revelará el Indicador de Compromiso (IoC) característico del envenenamiento ARP:

```
? (192.168.56.1) at 08:00:27:aa:bb:cc [ether] on eth0   <-- MAC de Kali
? (192.168.56.100) at 08:00:27:aa:bb:cc [ether] on eth0 <-- MAC de Kali
```

Firma defensiva: Se observa exactamente la **misma dirección MAC física asignada a dos direcciones IP distintas** en el mismo segmento de red.

5.2. Mapeo en el marco MITRE ATT&CK

Esta actividad se clasifica bajo los siguientes identificadores tácticos:

- **Táctica** : Credential Access / Collection (Acceso a Credenciales y Recolección).
- **Técnica** : Adversary-in-the-Middle: ARP Cache Poisoning (Código MITRE **T1557.002**).

6. Estrategias de Mitigación Defensiva.

Para neutralizar este vector de ataque de forma integral, el equipo defensivo debe aplicar controles en dos niveles estructurales: desarrollo de aplicaciones web y arquitectura de red local.

- Uso Obligatorio de HTTPS (TLS), Cifra todo el tráfico HTTP (puerto 443). Impide que herramientas como Wireshark lean la cabecera Cookie: en texto plano.
- Bandera HttpOnly,Configura la cookie (Set-Cookie: ...; HttpOnly) para evitar su lectura mediante scripts del lado del cliente.
- Bandera Secure,"Restringe el envío de la cookie únicamente a conexiones cifradas HTTPS. Si se navega sobre HTTP, el navegador no la transmite."
- Dynamic ARP Inspection (DAI),Característica de switches administrados que valida los paquetes ARP entrantes cotejándolos con una base de datos de confianza.
- DHCP Snooping,Construye la base de datos de vinculación (IP-MAC-Puerto físico) que alimenta y permite el funcionamiento de DAI.

🧠 Sin rodeos:

1. ¿Cómo descubrimos la anomalía con arp -a?

Es como si abrieras tu libreta de contactos y notaras que dos personas completamente distintas tienen anotado **exactamente el mismo número de teléfono celular** . En cuanto ves dos nombres apuntando a la misma línea, sabes de inmediato que las comunicaciones están siendo desviadas.

2. ¿Cómo evitan las banderas de seguridad web el secuestro de sesión?

- **HTTPS + Bandera Secure** : Es como colocar la cookie dentro de una **caja fuerte blindada** . Aunque el tráfico sea interceptado en el trayecto, solo se observará texto cifrado ilegible.
- **Bandera HttpOnly** : Es como colocarle un sello a la cookie que indica **"Solo para validación del servidor"** , evitando que código no autorizado en la página pueda leerla.

7. Conclusiones y Recomendaciones Operativas para el SOC.

El laboratorio práctico de **ARP Spoofing y Session Hijacking** demuestra la fragilidad de las arquitecturas de red locales que carecen de controles de inspección en la Capa 2 y de cifrado en la Capa 7.
Como recomendaciones operativas para la postura defensiva del SOC:

1. **Auditoría Continua de Capa 2** : Desplegar sondas NIDS (Snort/Suricata) con reglas de detección de respuestas ARP no solicitadas o cambios repentinos de asignación MAC-IP.
2. **Hardening de Servidores Web** : Auditar la configuración de servidores de aplicaciones para garantizar que todas las galletas de autenticación incorporen las directivas Secure, HttpOnly y SameSite=Strict.
3. **Migración Estricta a HTTPS** : Implementar políticas HSTS (HTTP Strict Transport Security) para forzar la navegación cifrada en todos los endpoints de la organización.